Bahan Belajar UTS Mobile Programming (Android, Java)

Disusun dari 5 sample code dosen dan 7 PPT (Sesi 1 sampai 7). Fokusnya **materi yang dipakai di sample code**. Dari PPT hanya diambil **teori yang penting**: definisi, arsitektur, lifecycle, dan konsep yang menjelaskan kode.

Sesi	Topik	Sample code
1	Introduction to Android (teori)	-
2	Multiple Activities and Intent	**02c**
3	Activity Lifecycle	**03c**
4	User Interface (layout yang dipakai di sample code)	dipakai di semua project
5	RecyclerView and Adapter	**05c**
6	Menu, Themes, Settings	**06a**
7	Data Persistence (SQLite)	**07c**

Cara pakai

- Ada waktu: baca Sesi 1 sampai 7 berurutan.
- Waktu mepet: baca **Rangkuman Kilat**, lalu kerjakan **Latihan Soal**.
- `SLIDE` = definisi yang hampir sama dengan kalimat di slide, paling mungkin keluar sebagai soal teori.
- `KODE` = penjelasan kode dosen.

Daftar Isi

SESI 1. Introduction to Android

1.1 Apa itu Android
1.2 Arsitektur Android
1.3 Activity dan Layout
1.4 Struktur project dan file penting

SESI 2. Multiple Activities and Intent (project 02c)

2.1 Aplikasinya ngapain
2.2 Setiap Activity wajib didaftarkan di Manifest
2.3 Intent
2.4 Pindah halaman dan kirim data
2.5 Buka browser (Implicit Intent)
2.6 Menangani klik banyak tombol
2.7 Toast
2.8 Layout 02c

SESI 3. Activity Lifecycle (project 03c)

3.1 Aplikasinya ngapain
3.2 Empat state Activity
3.3 Tujuh method lifecycle
3.4 Tiga lifetime
3.5 Urutan yang muncul di Logcat
3.6 Log vs Toast
3.7 Layout 03c: RelativeLayout

SESI 4. User Interface

4.1 View dan ViewGroup
4.2 Ukuran
4.3 Padding vs Margin, gravity vs layout_gravity
4.4 LinearLayout
4.5 RelativeLayout
4.6 ConstraintLayout dan FrameLayout (06a)

SESI 5. RecyclerView and Adapter (project 05c)

5.1 Aplikasinya ngapain
5.2 Komponen RecyclerView
5.3 Model / POJO (`Song.java`)
5.4 Adapter: 3 method WAJIB
5.5 ViewHolder
5.6 Memasang di MainActivity
5.7 Klik item: interface sebagai jembatan
5.8 Ringkasan langkah implementasi

SESI 6. Menu, Themes, Settings (project 06a)

6.1 Aplikasinya ngapain
6.2 Options Menu
6.3 Style dan Theme
6.4 Settings

SESI 7. Data Persistence (project 07c)

7.1 Aplikasinya ngapain
7.2 Pilihan penyimpanan data
7.3 Pembagian kelas di 07c
7.4 Tiga objek utama SQLite
7.5 SQLiteOpenHelper
7.6 Membuat tabel
7.7 Insert: `registerUser`
7.8 Query semua: `getAllUsers`
7.9 Query login: `getSpecificUser`
7.10 Update dan Delete (pelengkap CRUD, tidak ada di kode dosen)
7.11 OOP: constructor dan NullPointerException
7.12 Kelemahan kode 07c (bisa jadi soal analisis)

RANGKUMAN KILAT

Teori
Kode
Error klasik (soal "cari kesalahan")

LATIHAN SOAL (jawaban di bawah)

A. Pilihan ganda
B. Esai singkat
C. Cari kesalahan
D. Lengkapi kode
Kunci Jawaban

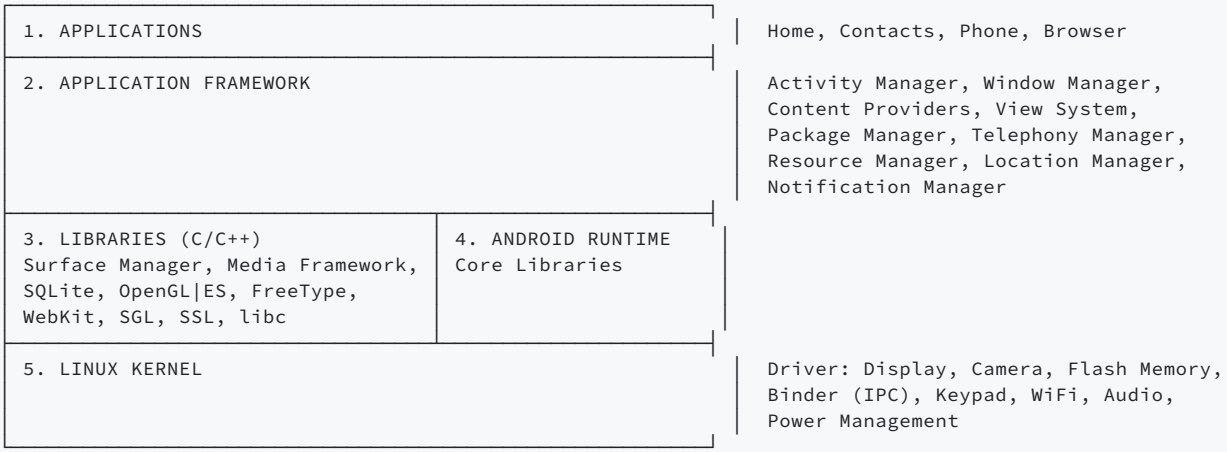
SESI 1. Introduction to Android

1.1 Apa itu Android

SLIDE Android adalah **sistem operasi mobile berbasis Linux yang dimodifikasi**. Dikembangkan oleh **Android, Inc.**, lalu **dibeli Google tahun 2005**. Aplikasinya dibuat dengan campuran **Java dan XML**.

1.2 Arsitektur Android PENTING

SLIDE OS Android dibagi menjadi **lima bagian dalam empat layer**. Libraries dan Android Runtime ada di layer yang sama, makanya 5 bagian tapi 4 layer.



Bagian	Fungsi
Applications	Aplikasi bawaan (Contacts, Calendar, Maps, browser) dan aplikasi buatan kita
Application Framework	API yang dipakai aplikasi untuk mengatur tampilan dan perilakunya. Kita memakai API yang sama dengan aplikasi bawaan
Libraries	Library C/C++ yang diakses lewat framework. Contoh: **SQLite** (database), **WebKit** (browser), **OpenGL ES** (grafis), **SSL** (keamanan)
Android Runtime	Core libraries yang mengimplementasikan bahasa Java. **Setiap aplikasi berjalan di prosesnya sendiri**
Linux Kernel	Paling bawah. Berisi **driver** hardware, serta layanan inti seperti **keamanan dan manajemen memori**

Contoh soal: "SQLite ada di layer apa?" → **Libraries**. "Driver kamera?" → **Linux Kernel**. "Activity Manager?" → **Application Framework**.

1.3 Activity dan Layout

SLIDE Dari slide:

- **Activity** = satu hal yang bisa dilakukan user, biasanya satu layar, ditulis dalam **Java**. "Activities define actions."
- **Layout** = tampilan layar, ditulis dalam **XML**. "Layout define how the user interface is presented."

KODE Di semua project dosen pasangannya selalu ada: `MainActivity.java` ↔ `activity_main.xml`. Keduanya disambungkan oleh `setContentView`:

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);         // wajib panggil versi induknya
        setContentView(R.layout.activity_main);     // pasang layout ke layar
        // setelah baris ini baru boleh findViewById
    }
}
```

1.4 Struktur project dan file penting

```
app/src/main/
├── AndroidManifest.xml      ← konfigurasi: daftar Activity, ikon, tema
├── java/com/example/.../    ← kode Java
└── res/                     ← resource (semua yang bukan kode)
    ├── layout/              ← tampilan (XML)
    ├── menu/                ← isi menu (06a)
    ├── values/              ← strings.xml, colors.xml, styles.xml, themes.xml
    ├── values-night/        ← versi dark mode (06a)
    └── xml/                 ← pengaturan Preferences (06a)
```

File Gradle (`build.gradle.kts`, `settings.gradle.kts`) di luar `app/` dipakai untuk meng-compile project dan mengunduh library.

Kelas `R` dan `findViewById` Setiap resource otomatis dapat ID di kelas `R`, misalnya `R.layout.activity_main`, `R.id.btnLogin`, `R.menu.menu_main`, `R.string.app_name`.

```
<Button android:id="@+id/btnLogin" ... />     <!-- + artinya buat ID baru -->
```

```
Button btnLogin = findViewById(R.id.btnLogin); // ambil SETELAH setContentView
```

`@+id/nama` membuat ID baru. `@id/nama` (tanpa +) merujuk ID yang sudah ada, contohnya di 03c: `android:layout_below="@id/tvHelloWorld"`.

strings.xml SLIDE Simpan teks di `strings.xml`, **jangan di-hardcode** di layout. Isinya pasangan nama/nilai String. Dipakai dengan `@string/nama` di XML. Teks jadi mudah diubah di satu tempat dan mudah diterjemahkan.

SESI 2. Multiple Activities and Intent (project 02c)

2.1 Aplikasinya ngapain

Layar pertama berisi kolom username, kolom password, tombol **LOGIN**, dan tombol **Register**.

- **LOGIN** → muncul Toast, lalu pindah ke layar kedua yang menulis "Halo [username]".
- **Register** → membuka Instagram di browser.

2.2 Setiap Activity wajib didaftarkan di Manifest

SLIDE Semua Activity harus dideklarasikan di `AndroidManifest.xml`. Kalau tidak, **sistem tidak tahu Activity itu ada**, dan aplikasi crash dengan `ActivityNotFoundException` saat Activity itu dibuka.

```
<application ...>
    <activity android:name=".MainActivity" android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <activity android:name=".MainActivity2" android:exported="false" />
</application>
```

- `android:name` wajib. Diawali titik karena digabung dengan nama package (`com.example.myapplication02c.MainActivity2`).
- `MAIN` + `LAUNCHER` = Activity yang dibuka pertama saat ikon aplikasi ditekan.
- `exported="true"` = boleh dibuka dari luar aplikasi (launcher HP termasuk "luar"), jadi Activity utama harus `true`.

2.3 Intent

SLIDE **Intent adalah sejenis pesan**, "an intent to do something". Kalau Activity A ingin menjalankan Activity B, A **mengirim intent ke Android**, lalu Android menjalankan B dan memberikan intent itu kepadanya. Dosen menamai variabelnya `niatPindahHalaman` dan `niatBrowser`.

	Explicit Intent	Implicit Intent
Menyebut	Nama kelas tujuan	Hanya **aksi** yang diinginkan
Contoh di 02c	`new Intent(this, MainActivity2.class)`	`new Intent(Intent.ACTION_VIEW, uri)`
Yang dibuka	Activity yang disebut, di aplikasi sendiri	Aplikasi lain yang mampu menangani aksi itu (browser, kamera, dan lain-lain)

SLIDE Di `new Intent(this, Tujuan.class)`: parameter pertama = asal intent (`this` = Activity sekarang), parameter kedua = kelas Activity tujuan.

2.4 KODE Pindah halaman dan kirim data

Pengirim (MainActivity):

```
String strUsername = etUsername.getText().toString();   // getText() bukan String, perlu
toString()

Intent niatPindahHalaman = new Intent(this, MainActivity2.class);
niatPindahHalaman.putExtra("keyUsername", strUsername); // titip data (kunci, nilai)
startActivity(niatPindahHalaman);
```

Penerima (MainActivity2):

```
String usernameLama = this.getIntent().getStringExtra("keyUsername");
tvGreetings.setText("Halo " + usernameLama);
```

- `putExtra(kunci, nilai)` menitipkan data ke intent. Nilainya bisa String, int, boolean, dan lain-lain.
- `getIntent()` mengambil intent yang menjalankan Activity ini, lalu `getStringExtra` membaca datanya.
- **Kunci harus persis sama.** Kalau beda (misalnya `"keyusername"`), hasilnya `null` dan layar menampilkan "Halo null".

2.5 KODE Buka browser (Implicit Intent)

```
Uri uri = Uri.parse("https://www.instagram.com");     // ubah String jadi Uri
Intent niatBrowser = new Intent(Intent.ACTION_VIEW, uri);
startActivity(niatBrowser);
```

`ACTION_VIEW` artinya "tampilkan ini". Karena isinya alamat web, Android membuka browser.

2.6 KODE Menangani klik banyak tombol

```
public class MainActivity extends AppCompatActivity implements View.OnClickListener {

    protected void onCreate(...) {
        btnLogin.setOnClickListener(this);     // this = Activity ini yang menangani klik
        btnRegister.setOnClickListener(this);
    }

    @Override
    public void onClick(View view) {           // semua klik masuk ke sini
        if (view.getId() == R.id.btnLogin) { ... }
        else if (view.getId() == R.id.btnRegister) { ... }
    }
}
```

Satu method `onClick` menangani semua tombol, dibedakan dengan `view.getId()`. Pola yang sama dipakai di 03c dan 07c.

Cara lain yang muncul di 05c adalah **lambda**: `btn.setOnClickListener(v -> { ... });`

2.7 Toast

```
Toast.makeText(this, "Login ditekan", Toast.LENGTH_SHORT).show();
```

Tiga parameter: context (`this`), pesan, durasi (`LENGTH_SHORT` sekitar 2 detik, `LENGTH_LONG` sekitar 3,5 detik). SLIDE Toast adalah umpan balik ringan yang **hilang otomatis**. **Tanpa `.show()` Toast tidak muncul.**

2.8 KODE Layout 02c

- **LinearLayout vertical** berisi 2 EditText dan sebuah **LinearLayout horizontal** berisi 2 Button. Layout di dalam layout ini disebut **nested layout**.

- Dua tombol masing-masing `layout_weight="1"` dengan `weightSum="2"`, jadi ruangnya dibagi rata (teori weight di Sesi 4).
- `android:hint` = teks petunjuk abu-abu yang hilang saat user mengetik.
- `inputType="textPassword"` = huruf disamarkan. `inputType="textEmailAddress"` = keyboard dengan tombol @.

> Catatan: `etPassword` ada di layout tapi tidak dipakai di Java. Pengecekan password sungguhan baru ada di 07c.

SESI 3. Activity Lifecycle (project 03c)

3.1 Aplikasinya ngapain

Dua Activity. Setiap method lifecycle mencetak pesan ke **Logcat** (misalnya "masuk create 1"). Klik teks "Hello World!" → pindah ke MainActivity2. Tujuannya supaya kamu bisa **melihat urutan method lifecycle dipanggil**.

SLIDE Activity di sistem dikelola sebagai **activity stack** (tumpukan). Activity yang baru dibuka ditaruh paling atas. Sejak tampil sampai tersembunyi, Activity melewati tahap-tahap yang disebut **activity life cycle**.

3.2 Empat state Activity PENTING

State	Kondisi
Active / Running	Di depan layar, di puncak stack
Paused	Kehilangan fokus tapi **masih terlihat** (tertutup sebagian). Masih hidup, tapi bisa dimatikan sistem kalau memori sangat kurang
Stopped	**Tertutup sepenuhnya** oleh Activity lain. Data masih ada tapi tidak terlihat, dan sering dimatikan sistem saat memori dibutuhkan
Killed / Destroyed	Dibuang dari memori. Kalau dibuka lagi harus dibuat ulang dari awal

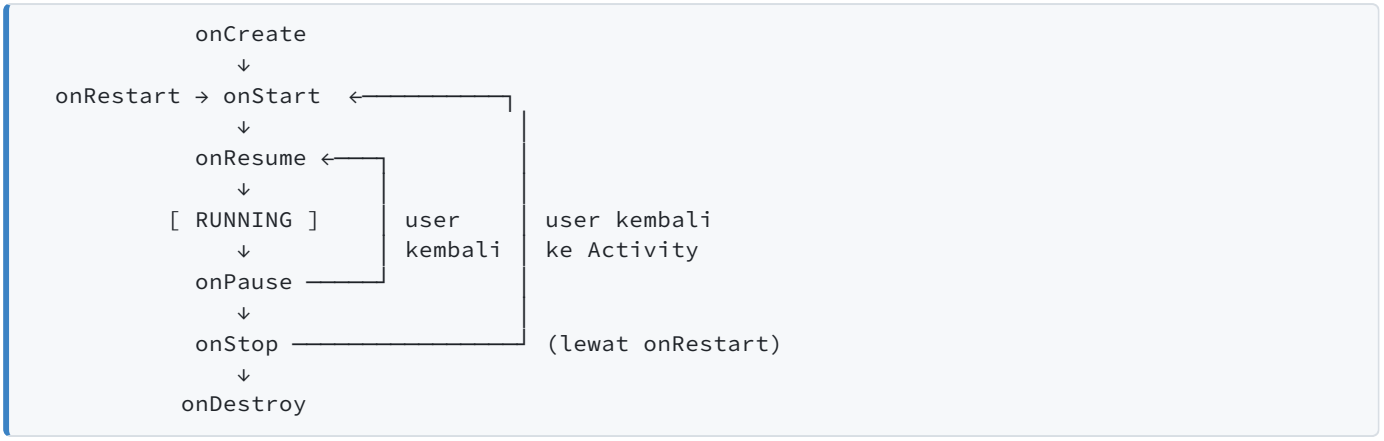
3.3 Tujuh method lifecycle

SLIDE Dari slide:

Method	Dipanggil saat
`onCreate()`	Activity **pertama kali dibuat**. Tempat `setContentView` dan `findViewById`
`onStart()`	Activity mulai **terlihat**
`onResume()`	Activity mulai **berinteraksi** dengan user (bisa disentuh)
`onPause()`	Activity kehilangan fokus, Activity lain akan di-resume
`onStop()`	Activity **tidak terlihat** lagi
`onDestroy()`	Sebelum Activity dihancurkan
`onRestart()`	Activity yang sudah stop **dibuka lagi** (dipanggil sebelum `onStart`)

Pasangannya simetris:

```
onCreate  ↔ onDestroy    (lahir / mati)
onStart   ↔ onStop       (terlihat / tidak terlihat)
onResume  ↔ onPause      (bisa disentuh / tidak)
```



KODE Di kode dosen setiap override memanggil `super.onXxx()` dulu, misalnya `super.onStart();`. Ini wajib.

3.4 Tiga lifetime PENTING

Lifetime	Rentang	Arti
Entire Lifetime	`onCreate()` sampai `onDestroy()`	Seluruh hidup Activity. Siapkan resource di onCreate, lepas di onDestroy
Visible Lifetime	`onStart()` sampai `onStop()`	Activity terlihat user. Bisa terjadi berkali-kali
Foreground Lifetime	`onResume()` sampai `onPause()`	Activity di depan dan berinteraksi dengan user. Sering berganti, jadi kodenya harus ringan

3.5 KODE Urutan yang muncul di Logcat PENTING

Kejadian	Urutan
Buka aplikasi	create 1 → start 1 → resume 1
Pindah ke Activity 2	**pause 1** → create 2 → start 2 → resume 2 → **stop 1**
Back dari Activity 2	pause 2 → (restart 1) → start 1 → resume 1 → stop 2 → **destroy 2**
Tekan Home	pause → stop (tidak destroy)
Kembali dari recent apps	(restart) → start → resume
Rotasi layar	pause → stop → destroy → create → start → resume

Catatan:

- Saat pindah, Activity 1 **pause dulu**, Activity 2 tampil, baru Activity 1 **stop**. Activity 1 **tidak di-destroy**, hanya disimpan di stack.
- Saat Back, Activity 2 di-destroy karena ditutup.
- `restart` tidak tercetak karena dosen tidak meng-override `onRestart`.
- Rotasi menghancurkan dan membuat ulang Activity, sebabnya data yang belum disimpan bisa hilang.

3.6 KODE Log vs Toast

```
String TAG = "MyApplication03c";
Log.v(TAG, "masuk create 1");
```

- `Log` mencetak ke **Logcat** di Android Studio, hanya untuk developer. TAG dipakai untuk memfilter pesan.
- Level log: `Log.v` verbose, `Log.d` debug, `Log.i` info, `Log.w` warning, `Log.e` error.

- `Toast` tampil ke **user**.

3.7 KODE Layout 03c: RelativeLayout

```
<TextView android:id="@+id/tvHelloWorld" ... />            <!-- pojok kiri atas (default) -->
<Button   android:id="@+id/btnLogin"
          android:layout_toRightOf="@id/tvHelloWorld" />     <!-- di kanan Hello World -->
<Button   android:id="@+id/btnRegister"
          android:layout_below="@id/tvHelloWorld" />        <!-- di bawah Hello World -->
<Switch   android:id="@+id/switchLampu"
          android:layout_toRightOf="@id/btnLogin"
          android:layout_below="@id/btnRegister" />          <!-- kanan btnLogin, bawah
btnRegister -->
```

Hasilnya kira-kira:

```
[Hello World!][ login ]
[ register ]
               [Saklar Lampu o]
```

- `Switch` = tombol geser on/off.
- Yang diberi listener adalah `tvHelloWorld` (TextView), bukan Button. Artinya **View apa pun bisa diklik** kalau diberi `setOnClickListener`.

SESI 4. User Interface

> Bagian ini membahas layout yang **dipakai di sample code**: LinearLayout (02c, 05c, 07c), RelativeLayout (03c), ConstraintLayout dan FrameLayout (06a).

4.1 View dan ViewGroup PENTING

SLIDE Dari slide:

- **Komponen GUI** (Button, TextView, EditText) adalah sejenis **View**, objek yang menempati ruang persegi di layar.
- **Layout** adalah sejenis View yang disebut **ViewGroup**, View khusus yang **bisa berisi View lain**.
- Saat aplikasi di-build, XML layout diubah menjadi **pohon View (view hierarchy)**.

```
LinearLayout (ViewGroup)        ← activity_main.xml di 02c
├── EditText (View)
├── EditText (View)
└── LinearLayout (ViewGroup)
    ├── Button (View)
    └── Button (View)
```

4.2 Ukuran

SLIDE `layout_width` dan `layout_height` **wajib** untuk semua layout dan View.

Nilai	Arti
`match_parent`	Sebesar parent-nya (penuh)
`wrap_content`	Cukup besar untuk menampung isinya
`dp`	Satuan ukuran umum (margin, padding, lebar) yang menyesuaikan kerapatan layar
`sp`	Seperti dp tapi ikut ukuran font user. **Khusus ukuran teks** (`textSize="20sp"` di 05c)

4.3 Padding vs Margin, gravity vs layout_gravity PENTING

	Arti
Padding	Jarak **di dalam** View, antara tepinya dan isinya
Margin	Jarak **di luar** View, antara View itu dan View lain
`gravity`	Posisi **isi** (misalnya teks) **di dalam** View
`layout_gravity`	Posisi **View itu sendiri** di dalam **wadahnya**

```
┌──────────── margin (luar) ────────────┐
│   ┌──── tepi View ──────────────┐     │
│   │  padding (dalam)            │     │
│   │     [ isi / teks ]          │     │
│   └─────────────────────────────┘     │
└───────────────────────────────────────┘
```

Nilai gravity yang umum: `top`, `bottom`, `left`, `right`, `center`, `center_vertical`, `center_horizontal`.

4.4 LinearLayout

SLIDE Menyusun View **berderet sesuai urutan di XML**. Arah ditentukan `android:orientation`: `vertical` (atas ke bawah) atau `horizontal` (kiri ke kanan).

Weight PENTING SLIDE `layout_weight` membagi **sisa ruang** secara proporsional. Contoh dari slide: View A weight 1, View B weight 2. Totalnya 3, jadi A dapat **1/3** dan B dapat **2/3** sisa ruang.

KODE Di 02c dua tombol masing-masing weight 1, jadi sisa ruang dibagi rata.

4.5 RelativeLayout

Posisi View ditentukan **relatif terhadap parent** atau **relatif terhadap View lain**. SLIDE Untuk relatif ke View lain, View acuan **harus punya ID**.

Relatif ke View lain	Relatif ke parent
`layout_above` = di atas View acuan	`layout_alignParentTop` / `Bottom` / `Left` / `Right`
`layout_below` = di bawah View acuan	`layout_centerInParent` = tengah parent
`layout_toLeftOf` = di kiri View acuan	`layout_centerHorizontal` = tengah horizontal
`layout_toRightOf` = di kanan View acuan	`layout_centerVertical` = tengah vertikal

KODE Contoh pemakaiannya di 03c (lihat 3.7).

4.6 ConstraintLayout dan FrameLayout (06a)

ConstraintLayout: View "diikat" ke tepi parent atau View lain.

```
<TextView
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

Diikat ke atas dan bawah sekaligus, juga kiri dan kanan sekaligus, hasilnya **tepat di tengah layar**.

FrameLayout: wadah sederhana. Di 06a dipakai sebagai tempat menempelkan Fragment settings.

Layout	Cara menyusun	Dipakai di
LinearLayout	Berderet satu arah	02c, 05c, 07c
RelativeLayout	Relatif ke parent atau View lain	03c
ConstraintLayout	Diikat ke parent atau View lain	06a
FrameLayout	Wadah untuk satu isi (misalnya Fragment)	06a

SESI 5. RecyclerView and Adapter (project 05c)

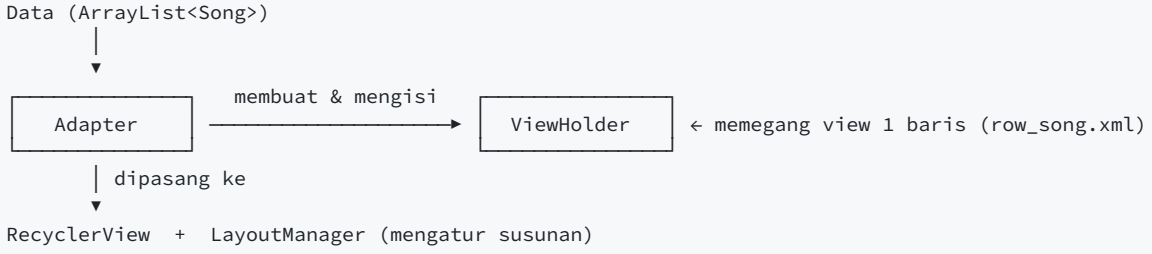
5.1 Aplikasinya ngapain

Menampilkan 100 lagu ("Judul 0 / Artis 0" sampai "Judul 99 / Artis 99") yang bisa di-scroll. Klik satu lagu → Toast "Judul X is clicked".

SLIDE **RecyclerView** adalah wadah yang bisa di-scroll untuk data dalam jumlah besar. **Efisien** karena memakai ulang View dalam jumlah terbatas. Kalau ada 100 data, yang dibuat hanya sebanyak yang terlihat di layar (misalnya 10 baris), lalu baris yang keluar layar **didaur ulang** untuk data berikutnya.

5.2 Komponen RecyclerView PENTING

Komponen	Di 05c	Fungsi
Data	`ArrayList<Song>`	Sumber data
RecyclerView	`rvSongs` di `activity_main.xml`	List yang bisa di-scroll
Layout 1 item	`row_song.xml`	Tampilan satu baris (judul 20sp, artis 14sp)
LayoutManager	`LinearLayoutManager`	Mengatur posisi item
Adapter	`SongAdapter`	SLIDE **Perantara antara data dan View**. Bukan bawaan Android, kita tulis sendiri
ViewHolder	`SongViewHolder`	SLIDE Menyiapkan View satu item untuk diisi data



5.3 KODE Model / POJO (`Song.java`)

POJO = *Plain Old Java Object*, kelas biasa yang isinya data.

```
public class Song {
    String title;
    String artist;

    public Song(String title, String artist) {   // constructor
        this.title = title;                       // this.title = atribut, title = parameter
        this.artist = artist;
    }
    public String getTitle() { return title; }   // getter
    public void setTitle(String title) { this.title = title; }  // setter
}
```

5.4 KODE Adapter: 3 method WAJIB PENTING

```
public class SongAdapter extends RecyclerView.Adapter<SongAdapter.SongViewHolder> {

    ArrayList<Song> songsInAdapter;

    // (1) Membuat ViewHolder dari row_song.xml. Dipanggil sebanyak baris yang terlihat
    @Override
    public SongViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.row_song, parent, false);
        return new SongViewHolder(view);
    }

    // (2) Mengisi data posisi ke-sekian ke ViewHolder. Dipanggil setiap baris muncul saat
scroll
    @Override
    public void onBindViewHolder(SongViewHolder holder, int position) {
        Song song = songsInAdapter.get(position);
        holder.tvSongTitle.setText(song.getTitle());
        holder.tvSongArtist.setText(song.getArtist());
    }

    // (3) Jumlah total data
    @Override
    public int getItemCount() {
        return songsInAdapter.size();
    }
}
```

Analogi formulir: `onCreateViewHolder` = mencetak formulir kosong, `onBindViewHolder` = mengisi formulir dengan data orang ke-sekian, `getItemCount` = memberi tahu ada berapa orang.

`LayoutInflater.inflate` mengubah file XML menjadi objek View. Parameter `false` artinya jangan langsung ditempel ke parent, karena RecyclerView yang menempelkannya.

5.5 KODE ViewHolder

```
public class SongViewHolder extends RecyclerView.ViewHolder {
    TextView tvSongTitle, tvSongArtist;

    public SongViewHolder(View itemView) {
        super(itemView);
        tvSongTitle = itemView.findViewById(R.id.tvSongTitle);   // dari itemView, bukan
Activity
        tvSongArtist = itemView.findViewById(R.id.tvSongArtist);
    }
}
```

ViewHolder **menyimpan referensi View** satu baris, jadi `findViewById` cukup dilakukan sekali per baris, tidak diulang setiap scroll.

5.6 KODE Memasang di MainActivity

```
// 1. Siapkan data
songs = new ArrayList<>();
for (int i = 0; i < 100; i++) {
    songs.add(new Song("Judul " + i, "Artis " + i));
}

// 2. Pasang LayoutManager dan Adapter
rvSongs = findViewById(R.id.rvSongs);
rvSongs.setLayoutManager(new LinearLayoutManager(this));   // WAJIB
rvSongs.setAdapter(songAdapter);
```

SLIDE Tiga LayoutManager bawaan:

- `LinearLayoutManager` → list (vertikal/horizontal)
- `GridLayoutManager` → grid, contoh `new GridLayoutManager(this, 2)` untuk 2 kolom
- `StaggeredGridLayoutManager` → grid dengan tinggi item berbeda-beda

Lupa LayoutManager atau Adapter → list kosong walaupun datanya ada.

> Tinggi root `row_song.xml` harus `wrap_content`. Kalau `match_parent`, satu baris memenuhi satu layar.

5.7 KODE Klik item: interface sebagai jembatan

Adapter tidak tahu harus berbuat apa saat item diklik. Yang memutuskan adalah Activity. Jembatannya adalah **interface**:

```
// Di SongAdapter
public interface OnItemClickListener {
    void onItemClick(Song song);
}

// Di onBindViewHolder
holder.itemView.setOnClickListener(v -> {
    if (listener != null) {
        listener.onItemClick(song);
    }
});
```

```
// Di MainActivity, isinya ditulis dengan lambda
SongAdapter songAdapter = new SongAdapter(songs, clickedSong -> {
    Toast.makeText(this, clickedSong.getTitle() + " is clicked", Toast.LENGTH_LONG).show();
});
```

Lambda `clickedSong -> { ... }` adalah cara singkat menulis `new SongAdapter.OnItemClickListener() { public void onItemClick(Song clickedSong) { ... } }`.

5.8 Ringkasan langkah implementasi

SLIDE Dari slide: (1) tambah dependency RecyclerView kalau perlu, (2) tambah RecyclerView ke layout, (3) buat layout satu item, (4) extend `RecyclerView.Adapter`, (5) extend `RecyclerView.ViewHolder`, (6) di `onCreate()` pasang adapter dan layout manager.

SESI 6. Menu, Themes, Settings (project 06a)

6.1 Aplikasinya ngapain

Layar utama menampilkan "Hello World!" besar, biru, tebal di tengah. Di app bar (bar atas) ada menu **Help**, **Settings**, **Log Out**. Klik Settings → membuka halaman pengaturan (signature, default reply, sync, attachment).

6.2 Options Menu

SLIDE Options menu ada di **pojok kanan app bar**. Item penting tampil sebagai ikon atau teks, sisanya masuk ke **overflow** (titik tiga). Dipakai untuk pindah ke Activity lain dan membuka pengaturan.

(Sebagai teori, slide menyebut 4 jenis menu: options menu, context menu, contextual action bar, popup menu. Yang dipakai di sample code hanya **options menu**.)

SLIDE **Tiga langkah:** (1) buat resource menu di `res/menu`, (2) override `onCreateOptionsMenu()`, (3) override `onOptionsItemSelected()`.

KODE **File menu (`res/menu/menu_main.xml`):**

```
<menu ...>
    <item android:id="@+id/menu_help"     android:title="Help"     app:showAsAction="always" />
    <item android:id="@+id/menu_settings" android:title="Settings" app:showAsAction="always" />
    <item android:id="@+id/menu_logout"   android:title="Log Out"  app:showAsAction="ifRoom" />
</menu>
```

`showAsAction`	Arti
`always`	Selalu tampil di app bar
`ifRoom`	Tampil di app bar kalau muat, kalau tidak masuk overflow (titik tiga)
`never`	Selalu di overflow (titik tiga)

KODE **Dua method di Activity:**

```
// (1) MEMBUAT menu dari menu_main.xml
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.menu_main, menu);
    return true;                        // true = tampilkan menunya
}

// (2) MENANGANI klik menu
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    int id = item.getItemId();
    if (id == R.id.menu_help) {
        Toast.makeText(this, "help di klik", Toast.LENGTH_SHORT).show();
        return true;                    // true = klik sudah ditangani
    } else if (id == R.id.menu_settings) {
        Intent niatSettings = new Intent(this, SettingsActivity.class);
        startActivity(niatSettings);
        return true;
    } else if (id == R.id.menu_logout) {
        ...
        return true;
    }
    return super.onOptionsItemSelected(item);   // klik lain serahkan ke induk
}
```

Polanya mirip `onClick` di 02c: satu method untuk semua item, dibedakan dengan id.

6.3 Style dan Theme PENTING

SLIDE **Style** = kumpulan properti tampilan (warna, ukuran font, padding, background, dan lain-lain) untuk **satu View**. Ditulis di XML terpisah dari layout. Filosofinya mirip **CSS**: memisahkan desain dari konten.

SLIDE **Theme** = style yang diterapkan ke **seluruh Activity atau aplikasi**, lewat atribut `android:theme` di manifest (`<application>` untuk seluruh aplikasi, `<activity>` untuk satu Activity).

Dua cara pewarisan style:

- Mewarisi style **bawaan Android** → pakai atribut `parent`.
- Mewarisi style **buatan sendiri** → SLIDE tidak perlu `parent`, cukup **awali nama dengan nama style induk, dipisah titik**.

KODE **`res/values/styles.xml` di 06a memakai kedua cara:**

```
<style name="MyJumboText" parent="Widget.AppCompat.TextView">      <!-- cara 1: parent -->
    <item name="android:textColor">@color/red</item>
    <item name="android:textSize">48sp</item>
    <item name="android:textStyle">bold</item>
</style>

<style name="MyJumboText.blue">                                    <!-- cara 2: prefix titik --
>
    <item name="android:textColor">@color/blue</item>
</style>
```

Dipakai di layout: `style="@style/MyJumboText.blue"`. **Hasilnya: biru, 48sp, bold.** `MyJumboText.blue` mewarisi semua atribut `MyJumboText`, lalu **menimpa** warna merah jadi biru. Ukuran dan bold tetap diwarisi.

Warna diambil dari `colors.xml`: `@color/blue` = `#FF0000FF`. Formatnya `#AARRGGBB`, AA = transparansi (FF = tidak transparan).

KODE **Theme di 06a:**

- `values/themes.xml` = theme mode terang. `values-night/themes.xml` = theme mode gelap, dipakai otomatis saat HP dalam dark mode.
- TEMUAN **Temuan (bagus untuk soal "kenapa"):** theme dark mode memakai `NoActionBar`. Akibatnya, **saat dark mode, action bar hilang dan menu Help/Settings/Log Out tidak muncul**. `SettingsActivity` juga diberi theme `NoActionBar` di manifest, sehingga `getSupportActionBar()` mengembalikan `null`. Itu sebabnya kode memakai `if (actionBar != null)` supaya tidak crash.

6.4 Settings

SLIDE Settings memungkinkan user mengatur perilaku aplikasi. Dipakai untuk nilai yang **jarang berubah**. Diakses lewat options menu atau navigation drawer.

KODE **SettingsActivity:**

```
public class SettingsActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.settings_activity);       // isinya FrameLayout id "settings"

        if (savedInstanceState == null) {                 // hanya saat pertama dibuat
            getSupportFragmentManager()
                .beginTransaction()
                .replace(R.id.settings, new SettingsFragment())  // tempel fragment ke
FrameLayout
                .commit();
        }

        ActionBar actionBar = getSupportActionBar();
        if (actionBar != null) {
            actionBar.setDisplayHomeAsUpEnabled(true);    // tampilkan panah back
        }
    }

    public static class SettingsFragment extends PreferenceFragmentCompat {
        @Override
        public void onCreatePreferences(Bundle savedInstanceState, String rootKey) {
            setPreferencesFromResource(R.xml.root_preferences, rootKey);
        }
    }
}
```

- **Fragment** = potongan layar yang ditempel di dalam Activity. **FrameLayout** jadi wadahnya.
- **FragmentTransaction**: `beginTransaction()` → `replace(wadah, fragment)` → `commit()`. Tanpa `commit()` tidak terjadi apa-apa.
- `if (savedInstanceState == null)`: saat layar diputar, Activity dibuat ulang (Sesi 3) dan Fragment lama dipulihkan otomatis. Pengecekan ini mencegah Fragment ditempel dua kali.
- **PreferenceFragmentCompat** otomatis membuat halaman settings dari XML dan **menyimpan pilihan user** ke SharedPreferences.

KODE **`res/xml/root_preferences.xml`:**

Elemen	Bentuk	Key
`PreferenceCategory` "Messages"	Judul kelompok	-
`EditTextPreference`	Isian teks "Your signature"	`signature`
`ListPreference`	Pilih "Reply" / "Reply to all"	`reply`
`PreferenceCategory` "Sync"	Judul kelompok	-
`SwitchPreferenceCompat`	Saklar "Sync email periodically"	`sync`
`SwitchPreferenceCompat` + `dependency="sync"`	Saklar yang **hanya aktif kalau sync menyala**	`attachment`

Atribut penting:

- SLIDE `key`: **wajib** untuk preference yang menyimpan nilai. Ini nama penyimpanannya di SharedPreferences.
- SLIDE `title`: nama setting yang dilihat user.
- SLIDE `defaultValue`: nilai awal. Sebaiknya diberikan untuk semua setting.
- `dependency`: preference ini hanya aktif kalau preference lain (key yang disebut) aktif.
- `entries` = teks yang dilihat user ("Reply to all"), `entryValues` = nilai yang disimpan ("reply_all"). Keduanya dari `arrays.xml`.

- `useSimpleSummaryProvider="true"` = nilai yang dipilih tampil di bawah judul.
- `summaryOn` / `summaryOff` = keterangan berbeda saat saklar on dan off.

Membaca nilai yang tersimpan (tidak ada di kode dosen, tapi wajar ditanyakan):

```
SharedPreferences prefs = PreferenceManager.getDefaultSharedPreferences(this);
boolean sync = prefs.getBoolean("sync", false);          // false = nilai default
String signature = prefs.getString("signature", "");
```

SESI 7. Data Persistence (project 07c)

7.1 Aplikasinya ngapain

Ada kolom username dan password, tombol **login**, tombol **register**, dan teks "users:".

- **Register** → menyimpan username dan password ke database di HP.
- **Login** → mengecek kombinasi username dan password di database. Toast menampilkan `true` atau `false`.
- **Klik teks "users:"** → menampilkan semua user (`1. budi`, `2. ani`, ...).

Data **tetap ada walaupun aplikasi ditutup**, berbeda dengan ArrayList di 05c yang hilang saat aplikasi ditutup.

7.2 Pilihan penyimpanan data

SLIDE Lima pilihan dari slide:

Pilihan	Untuk
Shared Preferences	Data primitif privat dalam pasangan **key-value** (dipakai Settings di 06a)
Internal Storage	File privat di memori perangkat
External Storage	File publik di penyimpanan bersama (SD card)
SQLite Database	Data **terstruktur** di database privat (dipakai di 07c)
Network Connection	Data di server web

SLIDE Untuk data relasional, database lebih efisien karena bisa di-query. Android memakai **SQLite**, dan database aplikasi **hanya bisa diakses aplikasi itu sendiri**.

7.3 KODE Pembagian kelas di 07c

```
MainActivity  ──pakai──►  UserDAO  ──pakai──►  MySQLiteOpenHelper  ──►  database "my_db"
  (tampilan)            (query SQL)           (membuat/membuka DB)
                           │
                           └── menghasilkan objek User (model)
```

Kelas	Peran
`User.java`	Model: id, username, password
`MySQLiteOpenHelper.java`	Membuat dan membuka database, mengurus versi
`UserDAO.java`	**DAO (Data Access Object)**: semua query tabel user dikumpulkan di sini
`MainActivity.java`	Tampilan. Hanya memanggil method DAO, tidak menulis SQL

Keuntungan DAO: Activity tidak perlu tahu SQL. Kalau cara penyimpanan berubah, cukup ubah DAO.

7.4 Tiga objek utama SQLite PENTING

Objek	Fungsi
SQLiteOpenHelper	Membuat dan mengelola database (kita extend kelas ini)
SQLiteDatabase	Akses ke database untuk menjalankan query
Cursor	Penunjuk untuk membaca hasil query baris per baris

SLIDE Pakai `getWritableDatabase()` untuk **menulis** (insert, update, delete) dan `getReadableDatabase()` untuk **membaca** (query). Keduanya mengembalikan `SQLiteDatabase`.

7.5 KODE SQLiteOpenHelper

SLIDE Helper butuh dua informasi untuk membuat database: **nama** dan **versi**.

```
public class MySQLiteOpenHelper extends SQLiteOpenHelper {
    static final String DB_NAME = "my_db";
    static final int DB_VERSION = 1;

    public MySQLiteOpenHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(UserDAO.CREATE_TABLE);     // membuat tabel
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) { }
}
```

- SLIDE `onCreate` dipanggil saat database **pertama kali dibuat**.
- SLIDE `onUpgrade` dipanggil saat database **perlu di-upgrade** (versi naik). Parameternya: database, versi lama, versi baru.
- `onCreate` milik helper **berbeda** dengan `onCreate` milik Activity. Namanya sama, tugasnya lain.

Bagaimana helper memutuskan PENTING

Kondisi	Yang terjadi
Database **belum ada**	Buat database, panggil `onCreate()`
Versi helper **lebih besar** dari versi database	Panggil `onUpgrade()`
Versi helper **lebih kecil** dari versi database	Panggil `onDowngrade()`
Versi **sama**	Tidak ada yang dipanggil, database sudah up to date

AWAS **Jebakan:** kalau `CREATE_TABLE` diubah (misalnya tambah kolom) tanpa menaikkan `DB_VERSION`, tabel tidak berubah karena `onCreate` tidak dipanggil lagi. Solusinya naikkan versi dan isi `onUpgrade` (misalnya `ALTER TABLE ms_user ADD COLUMN email TEXT`), atau uninstall aplikasi.

7.6 KODE Membuat tabel

```
static final String CREATE_TABLE = "CREATE TABLE IF NOT EXISTS ms_user ("
        + "id INTEGER PRIMARY KEY AUTOINCREMENT, "
        + "username TEXT UNIQUE, "
        + "password TEXT )";
```

- `PRIMARY KEY AUTOINCREMENT` → id terisi otomatis 1, 2, 3, dan seterusnya.
- `UNIQUE` → username tidak boleh kembar.
- Nama tabel dan kolom disimpan di konstanta `static final` supaya tidak salah ketik.
- SLIDE Tipe data SQLite: `INTEGER`, `TEXT`, `REAL`, `NUMERIC`, `BLOB`.
- SLIDE Konvensi Android: primary key dinamai `_id`. Kode dosen memakai `id`, jadi ini bisa jadi soal "apa yang tidak sesuai konvensi".

7.7 KODE Insert: `registerUser`

```
ContentValues cv = new ContentValues();       // "paket" data: nama kolom → nilai
cv.put("username", username);
cv.put("password", password);

SQLiteDatabase db = mySQLiteOpenHelper.getWritableDatabase();   // buka untuk MENULIS
long result = db.insert("ms_user", null, cv);
db.close();
return result;     // id baris baru, atau -1 kalau gagal
```

SLIDE Data dimasukkan sebagai pasangan nama/nilai ke **`ContentValues`** dengan `put()`. Kalau username sudah ada, insert melanggar `UNIQUE` dan hasilnya `-1`.

7.8 KODE Query semua: `getAllUsers`

```
SQLiteDatabase db = mySQLiteOpenHelper.getReadableDatabase();   // buka untuk MEMBACA
Cursor cursor = db.query("ms_user", null, null, null, null, null, null);  // SELECT * FROM
ms_user

if (cursor != null && cursor.moveToFirst()) {          // ke baris pertama; false kalau kosong
    int indexId = cursor.getColumnIndex("id");          // cari nomor kolom dari namanya
    int indexUsername = cursor.getColumnIndex("username");
    int indexPassword = cursor.getColumnIndex("password");

    do {
        int id = cursor.getInt(indexId);                // baca nilai baris saat ini
        String username = cursor.getString(indexUsername);
        String password = cursor.getString(indexPassword);
        users.add(new User(id, username, password));
    } while (cursor.moveToNext());                      // lanjut; false kalau habis
    cursor.close();
    db.close();
}
return users;
```

SLIDE **Cursor** menunjuk ke semua baris hasil query, dan dipakai untuk menelusuri baris dan kolomnya. Pakai `do...while` karena `moveToFirst()` sudah menaruh Cursor di baris pertama, jadi baris itu diproses dulu sebelum `moveToNext()`.

7.9 KODE Query login: `getSpecificUser`

```
String[] columns = { "id" };
String selection = "username = ? AND password = ?";
String[] selectionArgs = { username, password };

Cursor cursor = db.query("ms_user", columns, selection, selectionArgs, null, null, null);
// = SELECT id FROM ms_user WHERE username = ? AND password = ?

int cursorCount = cursor.getCount();
cursor.close();
return cursorCount > 0;     // ada barisnya → login benar
```

Urutan parameter `db.query(...)`:

```
query(table, columns, selection, selectionArgs, groupBy, having, orderBy)
       FROM   SELECT   WHERE     nilai untuk ?   GROUP BY HAVING  ORDER BY
```

`columns = null` artinya semua kolom.

Kenapa pakai `?`: nilai dari `selectionArgs` dimasukkan dengan aman sebagai data, bukan sebagai perintah SQL. Ini mencegah **SQL Injection**, yaitu user mengetik kode SQL (misalnya `' OR '1'='1`) di kolom input untuk membobol login.

7.10 Update dan Delete (pelengkap CRUD, tidak ada di kode dosen)

```
ContentValues cv = new ContentValues();
cv.put("password", "rahasia");
db.update("ms_user", cv, "username = ?", new String[]{ "budi" });   // UPDATE ... WHERE
username='budi'
db.delete("ms_user", "username = ?", new String[]{ "budi" });       // DELETE ... WHERE
username='budi'
```

7.11 KODE OOP: constructor dan NullPointerException

`User.java` punya 3 constructor (**overloading**: nama sama, parameter beda):

```
public User() { id = 0; username = ""; password = ""; }     // isi nilai default
public User(String username, String password) { ... }
public User(int id, String username, String password) { ... }
```

Contoh dari dosen di MainActivity:

```
userA = new User();            // objek benar-benar dibuat di RAM
Log.v("asd", userA.username);  // aman, username = ""
```

Kalau `userA = new User();` dihapus, `userA` bernilai `null`. Mengakses `userA.username` → **crash NullPointerException**. Pelajarannya: **variabel objek harus diisi `new ...` dulu sebelum dipakai.** Alasan yang sama membuat `findViewById` sebelum `setContentView` crash.

7.12 Kelemahan kode 07c (bisa jadi soal analisis)

1. Password disimpan apa adanya (plain text), seharusnya di-hash.
2. Hasil `registerUser` tidak dicek, jadi user tidak tahu kalau register gagal karena username kembar.
3. Di `getAllUsers`, `cursor.close()` dan `db.close()` hanya di dalam `if`. Kalau tabel kosong, keduanya tidak ditutup.
4. Di `getSpecificUser`, `db.close()` tidak dipanggil.
5. Input kosong tetap bisa di-register.
6. Primary key dinamai `id`, bukan `_id` sesuai konvensi.

RANGKUMAN KILAT

Teori

Topik	Inti
Android	OS mobile berbasis Linux modifikasi, Android Inc., dibeli Google 2005
Arsitektur	5 bagian dalam 4 layer: Applications, Application Framework, Libraries + Android Runtime, Linux Kernel
Activity vs Layout	Activity (Java) = apa yang dilakukan. Layout (XML) = tampilan
Intent	Sejenis pesan untuk menjalankan Activity atau aplikasi lain
Explicit vs Implicit	Explicit menyebut kelas tujuan. Implicit menyebut aksi
4 state	Running, Paused (masih terlihat), Stopped (tertutup), Killed
3 lifetime	Entire (Create–Destroy), Visible (Start–Stop), Foreground (Resume–Pause)
View vs ViewGroup	Komponen GUI = View. Layout = ViewGroup (berisi View lain)
Padding vs Margin	Padding di dalam, margin di luar
gravity vs layout_gravity	Posisi isi di dalam View / posisi View di wadahnya
layout_weight	Membagi sisa ruang secara proporsional
RecyclerView	Data, RecyclerView, layout item, LayoutManager, Adapter, ViewHolder
Adapter	Perantara data dan View, ditulis sendiri
3 LayoutManager	Linear, Grid, StaggeredGrid
Style vs Theme	Style untuk satu View. Theme untuk seluruh Activity/aplikasi
Pewarisan style	Bawaan pakai `parent`. Buatan sendiri pakai prefix titik
Preference `key`	Wajib, nama penyimpanan di SharedPreferences
5 penyimpanan	SharedPreferences, Internal, External, SQLite, Network
3 objek SQLite	SQLiteOpenHelper, SQLiteDatabase, Cursor
Versi DB	Belum ada → onCreate. Helper > DB → onUpgrade. Helper < DB → onDowngrade. Sama → tidak ada

Kode

Topik	Inti
Activity	`super.onCreate` → `setContentView` → `findViewById`
`@+id` vs `@id`	`+` membuat ID baru, tanpa `+` merujuk ID yang ada
Explicit Intent	`new Intent(this, Tujuan.class)` + `startActivity`
Kirim data	`putExtra("key", nilai)` ↔ `getIntent().getStringExtra("key")`
Implicit Intent	`new Intent(Intent.ACTION_VIEW, Uri.parse(url))`
Toast vs Log	Toast untuk user (`.show()`!), Log untuk developer (Logcat)
A ke B	pause A → create B → start B → resume B → stop A
Back dari B	pause B → restart A → start A → resume A → stop B → destroy B

Topik	Inti
Rotasi	pause → stop → destroy → create → start → resume
RelativeLayout	`layout_below`, `layout_toRightOf` (perlu ID acuan)
3 method Adapter	`onCreateViewHolder`, `onBindViewHolder`, `getItemCount`
RecyclerView	`setLayoutManager(...)` + `setAdapter(...)`
Klik item	interface listener di Adapter, diisi lambda di Activity
Options menu	`onCreateOptionsMenu` (inflate) + `onOptionsItemSelected` (klik)
showAsAction	`always`, `ifRoom`, `never`
Fragment	`beginTransaction().replace(wadah, fragment).commit()`
Insert	`ContentValues` + `getWritableDatabase().insert(...)`, -1 = gagal
Query	`getReadableDatabase().query(...)` → Cursor → `moveToFirst` / `moveToNext`
`?` di query	Mencegah SQL Injection
NullPointerException	Memakai objek yang masih `null`

Error klasik (soal "cari kesalahan")

1. `findViewById` sebelum `setContentView` → NullPointerException.
2. Toast tanpa `.show()` → tidak muncul.
3. Kunci `putExtra` ≠ kunci `getStringExtra` → nilai `null`.
4. Activity tidak didaftarkan di manifest → `ActivityNotFoundException`.
5. Lupa `super.onCreate(...)` → crash.
6. RecyclerView tanpa `setLayoutManager` → list kosong.
7. Root `row_song.xml` tingginya `match_parent` → satu item memenuhi satu layar.
8. FragmentTransaction tanpa `commit()` → Fragment tidak tampil.
9. Ubah struktur tabel tanpa menaikkan `DB_VERSION` → tabel tidak berubah.
10. Objek dipakai tanpa `new` → NullPointerException.

LATIHAN SOAL (jawaban di bawah)

A. Pilihan ganda

1. SQLite dalam arsitektur Android berada di layer...
   a. Linux Kernel b. Libraries c. Application Framework d. Applications
2. Driver kamera dan WiFi berada di layer...
   a. Linux Kernel b. Android Runtime c. Libraries d. Application Framework
3. Intent untuk membuka halaman web di browser termasuk...
   a. Explicit Intent b. Implicit Intent c. Pending Intent d. Broadcast
4. Method lifecycle saat Activity mulai bisa berinteraksi dengan user adalah...
   a. onCreate b. onStart c. onResume d. onRestart
5. Activity yang kehilangan fokus tapi masih terlihat berada di state...
   a. Running b. Paused c. Stopped d. Destroyed
6. Visible lifetime berlangsung antara...
   a. onCreate dan onDestroy b. onStart dan onStop c. onResume dan onPause d. onRestart dan onStop
7. Atribut untuk mengatur posisi **teks di dalam** sebuah EditText adalah...
   a. layout_gravity b. gravity c. layout_margin d. padding
8. View A weight 1 dan View B weight 3. Bagian sisa ruang untuk View B adalah...
   a. 1/3 b. 1/4 c. 3/4 d. 2/3
9. Satuan yang direkomendasikan untuk `textSize` adalah...
   a. px b. dp c. sp d. pt
10. Berikut adalah LayoutManager bawaan RecyclerView, **kecuali**...
    a. LinearLayoutManager b. GridLayoutManager c. StaggeredGridLayoutManager d. RelativeLayoutManager
11. Method Adapter yang mengisi data ke tiap baris adalah...
    a. onCreateViewHolder b. onBindViewHolder c. getItemCount d. onCreate
12. `app:showAsAction="ifRoom"` artinya...
    a. Selalu di app bar b. Selalu di overflow c. Di app bar kalau muat d. Hanya di landscape
13. Style yang diterapkan ke seluruh Activity atau aplikasi disebut...
    a. Style b. Theme c. Layout d. Preference
14. Penyimpanan untuk data primitif privat dalam bentuk key-value adalah...
    a. SQLite b. External Storage c. Shared Preferences d. Network
15. Versi di SQLite helper = 3, versi database di HP = 2. Method yang dipanggil...
    a. onCreate b. onUpgrade c. onDowngrade d. Tidak ada
16. `db.insert(...)` mengembalikan -1 artinya...
    a. Berhasil di baris 1 b. Gagal c. Tabel kosong d. Database tertutup
17. Method untuk membuka database agar bisa **membaca** data adalah...
    a. getWritableDatabase b. getReadableDatabase c. execSQL d. moveToFirst

B. Esai singkat

18. Sebutkan 5 bagian arsitektur Android dan satu fungsi masing-masing.
19. Jelaskan beda Explicit Intent dan Implicit Intent, beserta contoh kodenya dari project 02c.
20. Tulis urutan method lifecycle saat user pindah dari MainActivity ke MainActivity2, lalu menekan Back.
21. Sebutkan dan jelaskan 3 lifetime Activity.
22. Apa beda padding dan margin? Apa beda gravity dan layout_gravity?
23. Sebutkan komponen RecyclerView dan 3 method wajib Adapter beserta fungsinya.

24. Apa fungsi `onCreateOptionsMenu` dan `onOptionsItemSelected`?
25. Style `MyJumboText.blue` dipakai pada TextView. Sebutkan warna, ukuran, dan ketebalan teksnya, beserta alasannya.
26. Jelaskan kapan `onCreate`, `onUpgrade`, dan `onDowngrade` milik SQLiteOpenHelper dipanggil.
27. Kenapa query login memakai `"username = ? AND password = ?"` dan bukan menyambung string langsung?
28. Untuk apa pengecekan `if (savedInstanceState == null)` di SettingsActivity?

C. Cari kesalahan

29.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Button btn = findViewById(R.id.btnLogin);
    btn.setOnClickListener(this);
    setContentView(R.layout.activity_main);
}
```

30.

```
// MainActivity
intent.putExtra("keyUsername", "budi");
// MainActivity2
String nama = getIntent().getStringExtra("username");
tvGreetings.setText("Halo " + nama);
```

31.

```
rvSongs = findViewById(R.id.rvSongs);
rvSongs.setAdapter(songAdapter);
// list tidak muncul, kenapa?
```

32. Developer menambah kolom `email` di `CREATE_TABLE`, lalu menjalankan ulang aplikasi tanpa uninstall. Insert email gagal karena kolom tidak ditemukan. Kenapa, dan bagaimana memperbaikinya?

D. Lengkapi kode

33. Membuka `https://www.google.com` di browser:

```
Uri uri = ________("https://www.google.com");
Intent niat = new Intent(________, uri);
________(niat);
```

34. Menyimpan user baru ke tabel `ms_user`:

```
ContentValues cv = new ContentValues();
cv.________("username", username);
SQLiteDatabase db = mySQLiteOpenHelper.________();
long result = db.________("ms_user", null, cv);
```

35. Menampilkan options menu:

```
@Override
public boolean ________(Menu menu) {
    getMenuInflater().________(R.menu.menu_main, menu);
    return true;
}
```

Kunci Jawaban

A. Pilihan ganda

1. b
2. a
3. b
4. c
5. b
6. b
7. b
8. c (3 dari total 1+3 = 4)
9. c
10. d
11. b
12. c
13. b
14. c
15. b
16. b
17. b

B. Esai

18. Applications: aplikasi bawaan dan aplikasi kita. Application Framework: API untuk membangun aplikasi (Activity Manager, dan lain-lain). Libraries: library C/C++ seperti SQLite dan WebKit. Android Runtime: core libraries Java, setiap aplikasi di proses sendiri. Linux Kernel: driver hardware, keamanan, manajemen memori.
19. Explicit menyebut kelas tujuan: `new Intent(this, MainActivity2.class)`. Implicit hanya menyebut aksi, lalu Android mencari aplikasi yang bisa menanganinya: `new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.instagram.com"))`.
20. Pindah: pause 1 → create 2 → start 2 → resume 2 → stop 1. Back: pause 2 → restart 1 → start 1 → resume 1 → stop 2 → destroy 2.
21. Entire lifetime (onCreate sampai onDestroy, seluruh hidup Activity). Visible lifetime (onStart sampai onStop, Activity terlihat, bisa berkali-kali). Foreground lifetime (onResume sampai onPause, Activity di depan dan bisa disentuh, kodenya harus ringan).
22. Padding = jarak di dalam View antara tepi dan isinya. Margin = jarak di luar View ke View lain. Gravity = posisi isi di dalam View. layout_gravity = posisi View di dalam wadahnya.
23. Komponen: data, RecyclerView, layout satu item, LayoutManager, Adapter, ViewHolder. Method Adapter: `onCreateViewHolder` membuat ViewHolder dari layout item, `onBindViewHolder` mengisi data ke ViewHolder sesuai posisi, `getItemCount` memberi tahu jumlah data.
24. `onCreateOptionsMenu` meng-inflate file menu XML supaya menu tampil di app bar. `onOptionsItemSelected` menangani klik item menu, dibedakan dengan `item.getItemId()`.
25. Biru, 48sp, bold. `MyJumboText.blue` mewarisi `MyJumboText` (merah, 48sp, bold) lewat prefix titik, lalu menimpa warnanya menjadi biru.
26. `onCreate` saat database belum ada dan dibuat pertama kali. `onUpgrade` saat versi di helper lebih besar dari versi database. `onDowngrade` saat versi di helper lebih kecil. Kalau versinya sama, tidak ada yang dipanggil.
27. Mencegah SQL Injection. Nilai di `selectionArgs` dimasukkan sebagai data, bukan sebagai perintah SQL.
28. Mencegah Fragment ditambahkan dua kali saat Activity dibuat ulang (misalnya saat rotasi), karena Fragment lama sudah dipulihkan otomatis.

C. Cari kesalahan

29. `findViewById` dipanggil sebelum `setContentView`, jadi `btn` bernilai `null` dan `btn.setOnClickListener` crash (NullPointerException). Pindahkan `setContentView` tepat setelah `super.onCreate`.
30. Kunci tidak sama (`"keyUsername"` vs `"username"`). `nama` jadi `null`, layar menampilkan "Halo null".
31. Belum memasang LayoutManager. Tambahkan `rvSongs.setLayoutManager(new LinearLayoutManager(this));`.
32. Database sudah ada dan versinya sama, jadi `onCreate` tidak dipanggil lagi dan tabel lama tidak berubah. Perbaikannya: naikkan `DB_VERSION` dan tambahkan `ALTER TABLE ms_user ADD COLUMN email TEXT` di `onUpgrade`, atau uninstall aplikasi.

D. Lengkapi kode

33. `Uri.parse`, `Intent.ACTION_VIEW`, `startActivity`.
34. `put`, `getWritableDatabase`, `insert`.
35. `onCreateOptionsMenu`, `inflate`.

Semangat UTS-nya!